

Preuve de concept : Reporting

Projet : Preuve de concept (l'allocation de lits d'hôpital pour les urgences)

Client : Consortium MedHead

Table des matières

Table des matières

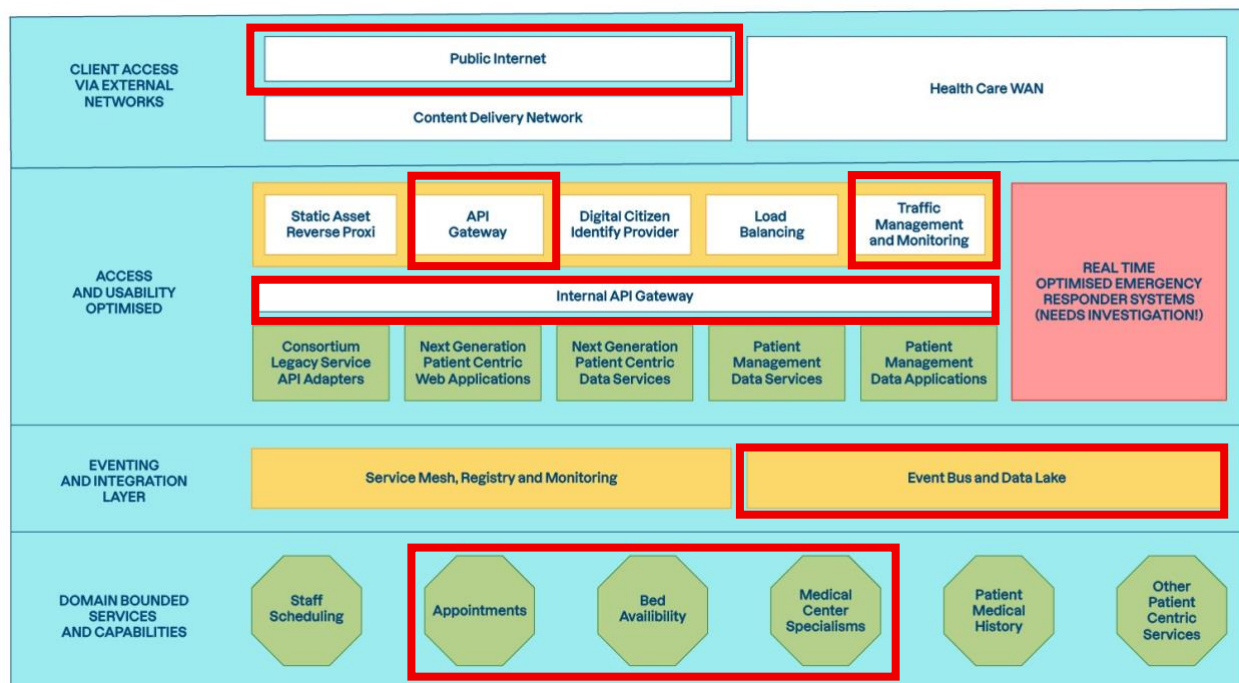
<i>Preuve de concept : Reporting</i>	1
<i>Table des matières</i>	1
<i>Architecture de la POC</i>	2
<i>Technologies retenues</i>	4
Langages de développement.....	4
Librairies et dépendances externes.....	4
<i>Normes et principes</i>	5
<i>Résultats</i>	6

Architecture de la POC

L'objet de la preuve de concept est de démontrer la faisabilité de l'architecture cible.

Celle-ci comporte 4 niveaux principaux dans son organisation, la POC doit donc fournir au moins un composant dans chaque couche pour prouver la faisabilité globale de l'architecture.

Les éléments retenus dans la POC sont encadrés sur diagramme ci-dessous :

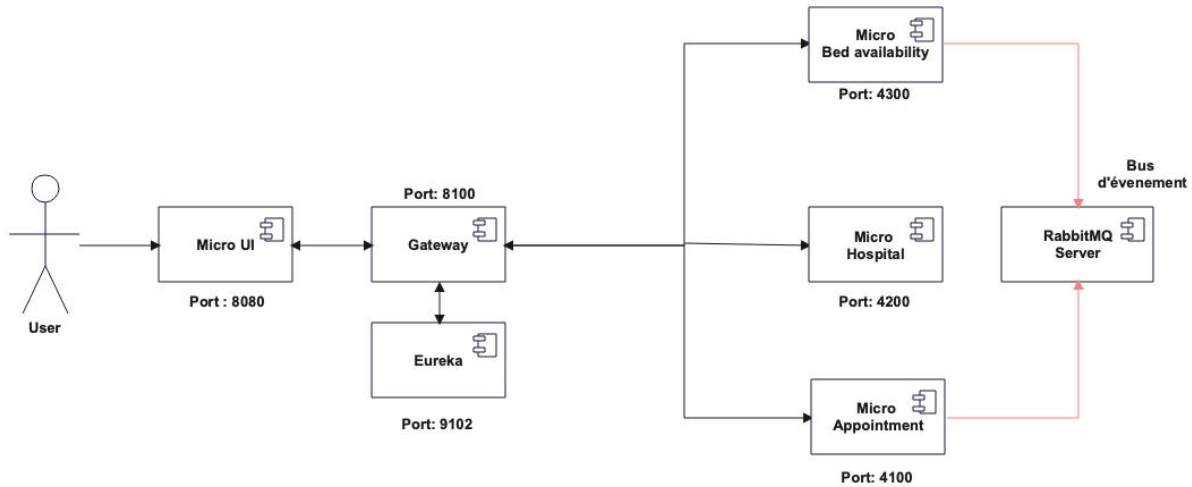


Les éléments constitutifs de la POC par niveau sont les suivants :

- Accès client via des réseaux externes : **un micro-service** présentant une interface simple
- Couche optimisée pour l'accès et l'utilisation :
 - un micro-service Gateway assurant le routage entre les micro-services et la sécurité
 - un micro-service permettant la surveillance des micro-services
- Couche d'événements et d'intégration : **un micro-service serveur RabbitMQ**
- Services et capacités limités au domaine :
 - un micro-service Hospital : celui-ci fournit la liste des spécialités médicales, la liste des hopitaux avec leur adresse, leurs spécialités, et la capacité maximale en lits
 - un micro-service Appointment : gère la liste des réservations de lits en cours, est connecté au bus d'évènement pour enregistrer des nouvelles réservations

- un micro-service Bed-Availability : cherche l'hôpital disponible le plus proche en fonction de l'adresse et des lits disponibles. Il est connecté au bus d'événement et envoie une demande de réservation

L'organisation de ces composants est présentée sur le schéma ci-dessous :



Technologies retenues

Langages de développement

Conformément aux exigences définies pour la POC, le langage Java est retenu pour les micro-services et l'interface utilisateur est développée avec le framework Angular.

Ces langages sont retenus pour leur robustesse, fiabilité, sécurité, évolutivité.

La version 22 de java avec Spring boot est utilisée.

Les données sont stockées sur un serveur MySQL.

RabbitMQ est utilisé pour la gestion du bus d'évènement.

Librairies et dépendances externes

Des librairies et dépendances externes ont été nécessaires pour la réalisation, celles-ci sont présentées ci-dessous :

Spécialités médicales et liste des hopitaux : les données sont récupérées via l'API publique FHIR française : <https://gateway.api.esante.gouv.fr/fhir>. Une clé de connexion à l'API a été créé pour la POC. Une dépendance MAVEN est disponible afin d'interagir facilement avec les données renvoyées par l'API.

Calcul de distance : la dépendance GraphHopper <https://www.graphhopper.com/> a été utilisée. Celle-ci permet le calcul de trajet entre 2 coordonnées latitude/longitude sur la base d'une carte Openstreetmap

Géolocalisation : pour déterminer les coordonnées latitude/longitude d'une adresse l'API publique <https://api-adresse.data.gouv.fr/> est utilisée.

Normes et principes

La réalisation de la Poc tient compte des normes et principes architecturaux suivants :

- RGPD : l'accès aux données est sécurisé par authentification au niveau de la Gateway. La POC n'utilise aucune donnée privée d'un utilisateur.
- Modèle d'architecture hexagonal/ports et adaptateurs : les micro-services métier ont été développés en suivant ce modèle d'organisation. L'évolutivité du code en sera facilitée
- Principe B1 - Continuité des activités des systèmes critiques pour les patients : l'architecture micro-service avec le monitoring via le serveur Eureka permettra d'assurer un haut niveau de continuité de service. La gestion de la charge des micro-services pourra être mise en place facilement.
- Principe B2 - Clarté grâce à une séparation fine des préoccupations : chaque micro-service a un domaine bien défini et limité, les données sont stockées dans des bases de données spécifiques à chaque micro-service
- Principe B3 - Intégration et livraison continues : la POC dispose d'un pipeline CI/CD permettant l'intégration et la livraison continué.
- Principe B4 - Tests automatisés précoces, complets et appropriés : chaque micro-service dispose des tests unitaires et d'intégration
- Principe B5 - Sécurité de type « shift-left » : l'accès au micro-services est sécurisé
- Principe B6 - Possibilité d'extension grâce à des fonctionnalités pilotées par les événements : un bus d'évènement via un serveur RabbitMQ est mis en place, la publication d'un évènement et sa consommation est fonctionnelle.

Limites

Les données concernant les hopitaux et la cartographie ont été limités à l'Alsace pour la POC. Une adresse en dehors de cette région générera une erreur.

Résultats

La POC fonctionne conformément au scénario décrit dans les exigences :

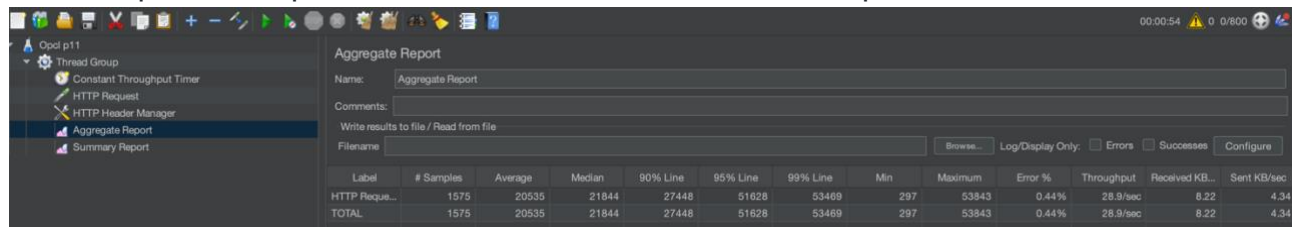
Par exemple, SUPPOSONS trois hôpitaux, comme suit :

Hôpital	Lits disponibles	Spécialisations
Hôpital Fred Brooks	2	Cardiologie, immunologie
Hôpital Julia Crusher	0	Cardiologie
Hôpital Beverly Bashir	5	Immunologie, neuropathologie, diagnostic

ET un patient nécessitant des soins en cardiologie,
QUAND un patient demande des soins en cardiologie ET que l'urgence est localisée près de l'hôpital Fred Brooks,
ALORS l'hôpital Fred Brooks devrait être proposé,
ET un événement devrait être publié pour réserver un lit.

Le délai de réponse d'une requête individuelle est de l'ordre de 200ms.

Un test de charge au taux nominal de 800 requêtes par secondes donne les résultats suivants pour un déploiement avec seulement une instance par microservice.



The screenshot shows the JMeter Aggregate Report window. The left sidebar lists the test components: Opd p11, Thread Group, Constant Throughput Timer, HTTP Request, HTTP Header Manager, Aggregate Report (selected), and Summary Report. The main area displays the 'Aggregate Report' for the 'Aggregate Report' test. It includes fields for Name, Comments, and a 'Write results to file / Read from file' section with a 'Browse...' button. Below these are checkboxes for 'Log/Display Only', 'Errors', 'Successes', and a 'Configure' button. A table at the bottom provides performance metrics for the 'HTTP Reque...' and 'TOTAL' samples.

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB...	Sent KB/sec
HTTP Reque...	1575	20535	21844	27448	51628	53469	297	53843	0.44%	28.9/sec	8.22	4.34
TOTAL	1575	20535	21844	27448	51628	53469	297	53843	0.44%	28.9/sec	8.22	4.34

Soit un taux moyen de réponse de l'ordre de 20s. Ce niveau n'est pas suffisant par rapport aux niveaux demandés.

La scalabilité de l'architecture microservice devra être utilisée pleinement pour pouvoir atteindre les objectifs de temps réponse demandés par la nouvelle architecture.

Etapes supplémentaires pour étendre les capacités de la POC

Les composants de la POC peuvent évoluer afin de couvrir plus de fonctionnalités de l'architecture cible. Les fonctionnalités suivantes pourraient être implémentées :

- Le load balancing doit être implémenté dans la gateway afin de tester les niveaux de service atteints avec plusieurs instances de micro-services.
- Le monitoring des différentes instances de micro-services peut être amélioré en employant Zipkin qui permet la traçabilité des requêtes entre les micro-services.
- La connexion à eureka pourrait être sécurisée.
- Le pipeline pourrait être poussé à l'étape de déploiement sur docker.